Semana 9: Patrón Decorator (Profundización) + Facade

Curso: Patrones de Diseño de Software (PIS1003)
Corte: 2 — Semana 9
Modalidad: Presencial — Aprendizaje Basado en Proyectos (ABPr)
Docente: Eliecer Montero Ojeda
Proyecto Integrador: Sistema de Gestión Académica con Arquitectura Basada en Patrones

---

1. CONEXIÓN CON LA SEMANA 8

1.1. Recordatorio: ¿dónde estamos?

En la Semana 8 estudiamos **Composite** (jerarquías parte-todo) y **Decorator** (extensiones dinámicas). Hoy profundizamos en **Decorator** y lo combinamos con un nuevo patrón estructural: **Facade**.

Semana	Patrones	Pregunta que responden
7	Adapter, Bridge	¿Cómo traduzco interfaces? ¿Cómo desacoplo abstracción de implementación?
8	Composite, Decorator	¿Cómo trato hojas y ramas uniformemente? ¿Cómo añado responsabilidades dinámicamente?
9	**Decorator (profundización), Facade**	**¿Cómo aplico decoradores a reportes y validaciones? ¿Cómo simplifico el acceso a subsistemas complejos?**

1.2. Los dos problemas que resolveremos hoy

Problema A — Extensión de reportes académicos: Un reporte de notas puede necesitar encabezado institucional, pie de página con firma, marca de agua "COPIA", y notas al margen con observaciones. ¿Cómo añadimos estos elementos sin crear 2^n combinaciones de reportes?

Problema B — Acceso simplificado al subsistema académico: El sistema tiene módulos de matrícula, notas, asistencia, reportes y notificaciones. Un cliente (por

ejemplo, el portal estudiantil) necesita realizar operaciones comunes (consultar promedio, inscribir materia, generar boletín) sin conocer la complejidad interna de cada módulo. ¿Cómo proporcionamos una **interfaz unificada**?

Decorator resuelve el Problema A. Facade resuelve el Problema B.

BLOQUE 2 — DECORATOR PROFUNDIZADO: REPORTES Y VALIDACIONES

2.1. Recordatorio del propósito

Intención GoF: *"Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality."*

2.2. Motivación: el reporte académico

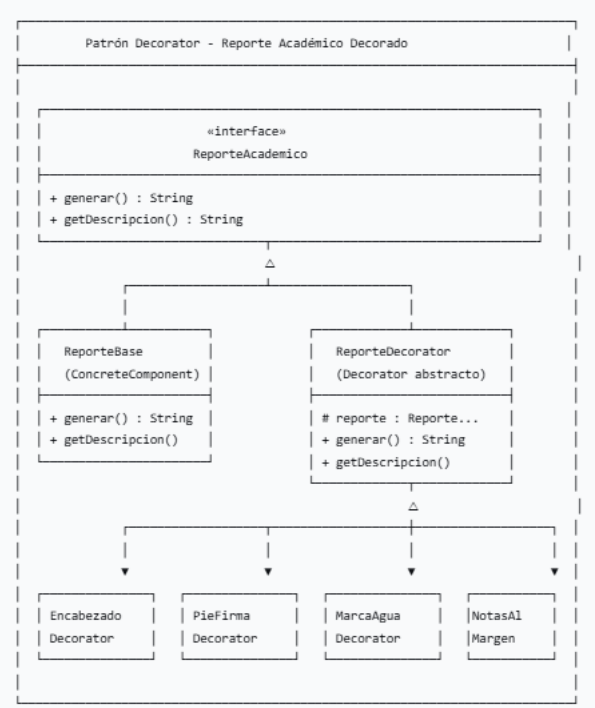
En el Sistema de Gestión Académica, un **reporte de notas** puede ser solicitado en contextos diferentes:

Contexto	Elementos que necesita
Reporte para el docente	Notas por estudiante
Reporte oficial para secretaría	+ Encabezado institucional + Pie con firma
Reporte para archivo	+ Marca de agua "COPIA"
Reporte con observaciones	+ Notas al margen
Reporte completo	Todos los anteriores

Si modelamos esto con herencia: ReporteConEncabezado, ReporteConEncabezadoYPie, ReporteConEncabezadoYPieYMarcaAgua, etc. → **explosión combinatoria**.

Solución Decorator: Un ReporteBase que genera el contenido mínimo, y decoradores que se van anidando según se necesiten.

2.3. Estructura aplicada al proyecto



2.4. Implementación en el proyecto integrador

Component — Interfaz del reporte:

```java
package co.uts.academico.reportes;

public interface ReporteAcademico {
    String generar();
    String getDescripcion();
}
```

ConcreteComponent — Reporte base (notas crudas):

```java
package co.uts.academico.reportes;

import java.util.List;

public class ReporteNotasBase implements ReporteAcademico {
    private final String estudiante;
    private final List<Double> notas;
    private final double promedio;

    public ReporteNotasBase(String estudiante, List<Double> notas) {
        this.estudiante = estudiante;
        this.notas = notas;
        this.promedio = notas.stream().mapToDouble(Double::doubleValue).average().orElse(0);
    }

    @Override
    public String generar() {
        StringBuilder sb = new StringBuilder();
        sb.append("REPORTE DE NOTAS\n");
        sb.append("Estudiante: ").append(estudiante).append("\n");
        sb.append("Notas: ").append(notas).append("\n");
        sb.append(String.format("Promedio: %.2f\n", promedio));
        return sb.toString();
    }

    @Override
    public String getDescripcion() {
        return "Notas crudas de " + estudiante;
    }
}
```

Decorator abstracto:

```java
package co.uts.academico.reportes;

public abstract class ReporteDecorator implements ReporteAcademico {
    protected final ReporteAcademico reporte;

    protected ReporteDecorator(ReporteAcademico reporte) {
        this.reporte = reporte;
    }

    @Override
    public String generar() {
        return reporte.generar();  // Delega en el componente decorado
    }

    @Override
    public String getDescripcion() {
        return reporte.getDescripcion();  // Delega en el componente decorado
    }
}
```

ConcreteDecorator 1 — Encabezado institucional:

```java
package co.uts.academico.reportes;

public class EncabezadoInstitucional extends ReporteDecorator {
    private static final String ENCABEZADO =
        "======================================\n" +
        "  UNIVERSIDAD TECNOLÓGICA DE SANTANDER\n" +
        "  Facultad de Ingenierías\n" +
        "  Sistema de Gestión Académica\n" +
        "======================================\n\n";

    public EncabezadoInstitucional(ReporteAcademico reporte) {
        super(reporte);
    }

    @Override
    public String generar() {
        return ENCABEZADO + super.generar();
    }

    @Override
    public String getDescripcion() {
        return "Encabezado institucional + " + super.getDescripcion();
    }
}
```

ConcreteDecorator 2 — Pie con firma:

```java
package co.uts.academico.reportes;

public class PieConFirma extends ReporteDecorator {
    private final String firmante;

    public PieConFirma(ReporteAcademico reporte, String firmante) {
        super(reporte);
        this.firmante = firmante;
    }

    @Override
    public String generar() {
        return super.generar() +
            "\n\n_____\n" +
            "Firma: " + firmante + "\n" +
            "Fecha: " + java.time.LocalDate.now() + "\n";
    }

    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " + Firma de " + firmante;
    }
}
```

ConcreteDecorator 3 — Marca de agua:

```java
package co.uts.academico.reportes;

public class MarcaAgua extends ReporteDecorator {
    private final String texto;

    public MarcaAgua(ReporteAcademico reporte, String texto) {
        super(reporte);
        this.texto = texto;
    }

    @Override
    public String generar() {
        String contenido = super.generar();
        // Inserta la marca de agua en cada línea (simulación de marca de agua)
        return contenido.replaceAll("(?m)^", "[" + texto + "] ");
    }

    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " + Marca de agua '" + texto + "'";
    }
}
```

ConcreteDecorator 4 — Notas al margen:

```java
package co.uts.academico.reportes;

import java.util.ArrayList;
import java.util.List;

public class NotasAlMargen extends ReporteDecorator {
    private final List<String> observaciones = new ArrayList<>();

    public NotasAlMargen(ReporteAcademico reporte) {
        super(reporte);
    }

    public NotasAlMargen agregarObservacion(String obs) {
        observaciones.add(obs);
        return this;
    }

    @Override
    public String generar() {
        StringBuilder sb = new StringBuilder(super.generar());
        sb.append("\n--- OBSERVACIONES ---\n");
        for (String obs : observaciones) {
            sb.append("• ").append(obs).append("\n");
        }
        return sb.toString();
    }

    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " + " + observaciones.size() + " observaciones";
    }
}
```

2.5. Cliente — Composición dinámica de decoradores

```java
import java.util.List;

public class GeneradorReportes {
    public static void main(String[] args) {
        List<Double> notasAna = List.of(4.5, 4.2, 4.8, 4.0);

        // Caso 1: Reporte simple (solo para el docente)
        ReporteAcademico simple = new ReporteNotasBase("Ana García", notasAna);
        System.out.println("=== Reporte simple ===");
        System.out.println(simple.generar());

        // Caso 2: Reporte oficial (encabezado + firma)
        ReporteAcademico oficial = new PieConFirma(
            new EncabezadoInstitucional(
                new ReporteNotasBase("Ana García", notasAna)
            ),
            "Dr. Eliecer Montero"
        );
        System.out.println("\n=== Reporte oficial ===");
        System.out.println(oficial.generar());

        // Caso 3: Reporte completo (encabezado + firma + marca de agua + observaciones)
        ReporteAcademico completo = new NotasAlMargen(
            new MarcaAgua(
                new PieConFirma(
                    new EncabezadoInstitucional(
                        new ReporteNotasBase("Ana García", notasAna)
                    ),
                    "Dr. Eliecer Montero"
                ),
                "COPIA"
            )
        ).agregarObservacion("Estudiante destacada")
         .agregarObservacion("Recomendada para monitoría");

        System.out.println("\n=== Reporte completo ===");
        System.out.println(completo.generar());
        System.out.println("\nDescripción: " + completo.getDescripcion());
    }
}
```

2.6. Consecuencias de Decorator (profundización)

Ventajas adicionales:

Ventaja	Explicación
Composición flexible	El orden de los decoradores se decide en runtime. Se puede tener Encabezado(Pie(MarcaAgua(Base))) o MarcaAgua(Pie(Encabezado(Base))).

Ventaja	Explicación
Responsabilidades pequeñas	Cada decorador hace **una sola cosa**: EncabezadoInstitucional solo añade encabezado; no sabe nada de firmas ni marcas de agua.
Extensibilidad sin modificación	Agregar DecoradorNumeroPagina no requiere tocar ReporteNotasBase ni los demás decoradores.

Desventajas adicionales:

Desventaja	**Explicación**
Depuración compleja	Un stack de 5 decoradores anidados puede ser difícil de rastrear en el debugger.
Orden importa	MarcaAgua antes de Encabezado produce un resultado diferente a Encabezado antes de MarcaAgua.
Identidad	reporte instanceof ReporteNotasBase falla si reporte está decorado.

2.7. Conexión con el proyecto integrador — Semana 9

En el Sistema de Gestión Académica, Decorator ya no solo se aplica a la **matrícula**, sino también a:

Módulo	**Uso de Decorator**
Reportes	Encabezados, pies, marcas de agua, observaciones al margen
Validaciones de inscripción	ValidacionBase + ValidaPrerrequisitos + ValidaCupo + ValidaPago

Módulo	Uso de Decorator
Cálculo de notas	NotaBase + BonificacionParticipacion + CurvaExamen + PuntosExtra
Notificaciones	NotificacionBase + ConCopia + ConAdjunto + Urgente

---

BLOQUE 3 — PATRÓN FACADE

3.1. Propósito

Intención GoF: *"Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use."*

Problema que resuelve: Cuando un subsistema tiene muchas clases interdependientes y el cliente necesita orquestar múltiples llamadas para una operación común, el código cliente se vuelve complejo y frágil. Facade proporciona una **puerta de entrada simplificada**.

3.2. Motivación: el portal estudiantil

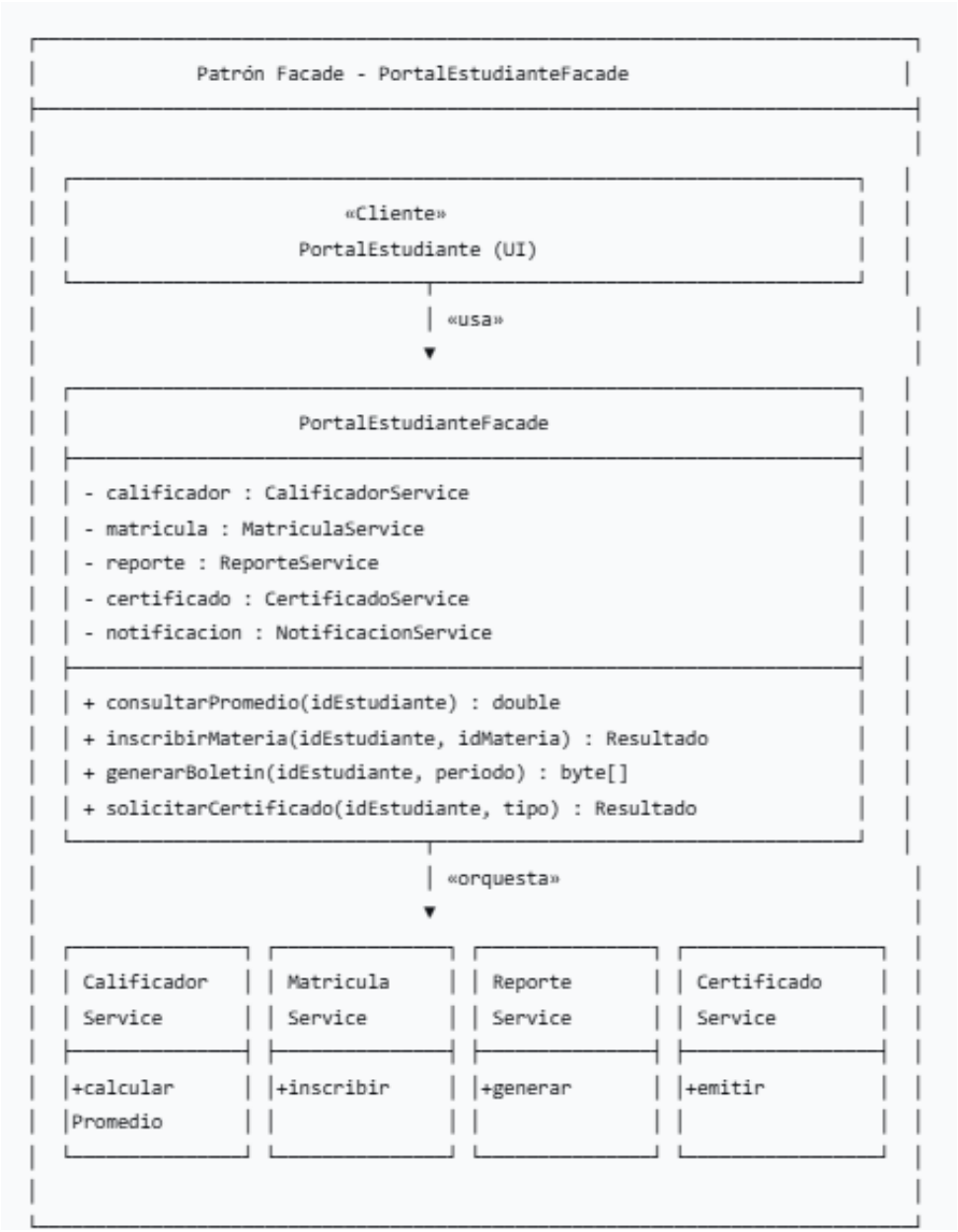
En el Sistema de Gestión Académica, un estudiante que ingresa al portal necesita realizar operaciones que internamente involucran múltiples módulos:

Operación del estudiante	**Módulos internos involucrados**
Consultar promedio	Calificador → NotaRepository → EstudianteRepository
Inscribir materia	MatriculaService → PrerrequisitoValidator → CupoManager → PagoGateway
Generar boletín	ReporteFactory → NotaRepository → ReporteRenderer → PDFExporter
Solicitar certificado	CertificadoService → FirmaDigital → NotificacionService

Sin Facade, el cliente tendría que conocer y orquestar todas estas clases.

Solución Facade: PortalEstudianteFacade expone métodos simples (consultarPromedio(), inscribirMateria(), generarBoletin(), solicitarCertificado()) que internamente coordinan los subsistemas.

3.3. Estructura aplicada al proyecto



3.4. Implementación en el proyecto integrador

Subsistema 1 — Servicio de calificaciones:

```
package co.uts.academico.subsistemas;

public class CalificadorService {
    public double calcularPromedio(int idEstudiante) {
        // Simula consulta a base de datos
        return 4.35;
    }

    public double calcularPromedioPorMateria(int idEstudiante, int idMateria) {
        return 4.2;
    }
}
```

Subsistema 2 — Servicio de matrícula:

```
public class MatriculaService {
    public Resultado inscribir(int idEstudiante, int idMateria) {
        // Verifica prerrequisitos, cupo, pago
        boolean prerrequisitosOk = verificarPrerrequisitos(idEstudiante, idMateria);
        boolean cupoOk = verificarCupo(idMateria);

        if (!prerrequisitosOk) return Resultado.error("Prerrequisitos no cumplidos");
        if (!cupoOk) return Resultado.error("Sin cupo disponible");

        return Resultado.exito("Inscripción exitosa");
    }

    private boolean verificarPrerrequisitos(int idEstudiante, int idMateria) {
        return true; // Simulado
    }

    private boolean verificarCupo(int idMateria) {
        return true; // Simulado
    }
}
```

Subsistema 3 — Servicio de reportes:

```
package co.uts.academico.subsistemas;

public class ReporteService {
    public byte[] generarBoletin(int idEstudiante, String periodo) {
        // Orquesta: obtener notas, renderizar, exportar PDF
        return new byte[0]; // Simulado
    }
}
```

Subsistema 4 — Servicio de certificados:

```java
package co.uts.academico.subsistemas;

public class CertificadoService {
    public Resultado emitir(int idEstudiante, String tipo) {
        // Firma digital, genera PDF, notifica
        return Resultado.exito("Certificado emitido");
    }
}
```

Subsistema 5 — Servicio de notificaciones:

```java
package co.uts.academico.subsistemas;

public class NotificacionService {
    public void enviar(String destinatario, String asunto, String mensaje) {
        System.out.println("Notificación enviada a " + destinatario);
    }
}
```

Facade — Portal del estudiante:

```java
package co.uts.academico.fachada;

import co.uts.academico.subsistemas.*;

public class PortalEstudianteFacade {
    private final CalificadorService calificador;
    private final MatriculaService matricula;
    private final ReporteService reporte;
    private final CertificadoService certificado;
    private final NotificacionService notificacion;

    public PortalEstudianteFacade() {
        this.calificador = new CalificadorService();
        this.matricula = new MatriculaService();
        this.reporte = new ReporteService();
        this.certificado = new CertificadoService();
        this.notificacion = new NotificacionService();
    }

    public double consultarPromedio(int idEstudiante) {
        return calificador.calcularPromedio(idEstudiante);
    }
```

```
    public Resultado inscribirMateria(int idEstudiante, int idMateria) {
        Resultado resultado = matricula.inscribir(idEstudiante, idMateria);
        if (resultado.isExito()) {
            notificacion.enviar("estudiante@uts.edu.co",
                "Inscripción exitosa",
                "Te has inscrito en la materia " + idMateria);
        }
        return resultado;
    }

    public byte[] generarBoletin(int idEstudiante, String periodo) {
        return reporte.generarBoletin(idEstudiante, periodo);
    }

    public Resultado solicitarCertificado(int idEstudiante, String tipo) {
        Resultado resultado = certificado.emitir(idEstudiante, tipo);
        if (resultado.isExito()) {
            notificacion.enviar("estudiante@uts.edu.co",
                "Certificado listo",
                "Tu certificado de " + tipo + " está disponible");
        }
        return resultado;
    }
}
```

Cliente — Portal estudiantil (simplificado):

```
package co.uts.academico.ui;

import co.uts.academico.fachada.PortalEstudianteFacade;

public class PortalEstudiante {
    public static void main(String[] args) {
        PortalEstudianteFacade portal = new PortalEstudianteFacade();
        int idEstudiante = 12345;

        // El cliente no conoce los subsistemas internos
        System.out.println("Promedio: " + portal.consultarPromedio(idEstudiante));

        var inscripcion = portal.inscribirMateria(idEstudiante, 101);
        System.out.println("Inscripción: " + inscripcion.getMensaje());

        portal.generarBoletin(idEstudiante, "2026-1");
        System.out.println("Boletín generado");

        var cert = portal.solicitarCertificado(idEstudiante, "ESTUDIOS");
        System.out.println("Certificado: " + cert.getMensaje());
    }
}
```

3.5. Relación Facade vs. Adapter (Semana 7)

Aspecto	Adapter	Facade
Propósito	Traducir una interfaz a otra	Simplificar el acceso a un subsistema
¿Cuántas clases envuelve?	1 (Adaptee)	Muchas (subsistema completo)
¿Cambia la interfaz?	Sí (la traduce)	Sí (la simplifica)
¿Añade comportamiento?	No	No (solo coordina)
Cliente ve	Una interfaz compatible	Una interfaz simplificada

Regla mnemotécnica:

- **Adapter** = traductor (hace que A parezca B)
- **Facade** = recepcionista (te da un punto único para gestionar todo)

3.6. Consecuencias de Facade

Ventajas:

Ventaja	Explicación
Simplificación	El cliente no necesita conocer la complejidad interna
Desacoplamiento	El cliente no depende de las clases del subsistema
Punto único de cambio	Si el subsistema cambia, solo se modifica el Facade

Ventaja	Explicación
Facilita el uso	Los métodos del Facade son de alto nivel y autoexplicativos

Desventajas:

Desventaja	Explicación
Puede convertirse en "God Object"	Si acumula demasiadas responsabilidades
No impide el acceso directo	El cliente aún puede usar las clases del subsistema si lo necesita
Riesgo de sobre-simplificación	Puede ocultar funcionalidades avanzadas que algunos clientes necesitan

3.7. Conexión con el proyecto integrador — Semana 9

En el Sistema de Gestión Académica, Facade se aplica en:

Facade	Subsistemas que coordina	Clientes
PortalEstudianteFacade	Calificaciones, matrícula, reportes, certificados, notificaciones	Portal estudiantil (React)
PortalDocenteFacade	Calificaciones, asistencia, reportes	Portal docente
AdminAcademicoFacade	Matrícula, programas, cursos, reportes globales	Módulo administrativo

---

BLOQUE 4 — COMPARATIVA Y RELACIÓN CON PATRONES ANTERIORES

4.1. Tabla comparativa: Composite vs. Decorator vs. Facade

Criterio	Composite	Decorator	Facade
Intención	Tratar uniformemente hojas y composiciones	Añadir responsabilidades dinámicamente	Simplificar acceso a subsistema
Estructura	Árbol (1..* hijos)	Cadena (1 componente)	Estrella (muchos subsistemas)
Número de componentes	Muchos	Uno	Muchos (subsistemas)
¿Cambia la interfaz?	No	No	Sí (la simplifica)
¿Añade comportamiento?	No	Sí	No (solo coordina)
Alternativa a	Herencia con if/else por tipo	Explosión combinatoria de subclases	Acoplamiento directo al subsistema
Ejemplo en proyecto	Facultad → Programa → Curso	Reporte con encabezado + firma	PortalEstudianteFacade

4.2. Combinación de los cinco patrones estructurales

Un diseño maduro del Sistema de Gestión Académica combina:

- **Composite:** Estructura Facultad → Programa → Curso → Estudiante

- **Decorator:** Reportes decorados (encabezado, firma, marca de agua) y matrícula con becas
- **Facade:** PortalEstudianteFacade para simplificar el acceso
- **Adapter:** Integración con pasarelas de pago heterogéneas
- **Bridge:** Notificaciones multicanal (email, SMS, push)

Regla mnemotécnica:

- **Composite** = árbol (trata hojas y ramas igual)
- **Decorator** = capas (añade sin cambiar)
- **Facade** = recepcionista (un punto de entrada)
- **Adapter** = traductor (cambia interfaz)
- **Bridge** = arquitecto (separa jerarquías)

4.3. Relación con los principios SOLID

Principio	**Composite**	**Decorator**	**Facade**
S (Single Responsibility)	Cada nodo tiene una responsabilidad	Cada decorador hace una cosa	El Facade coordina, no implementa lógica de negocio
O (Open/Closed)	Nuevos nodos sin modificar cliente	Nuevos decoradores sin modificar base	Nuevos subsistemas sin modificar cliente
L (Liskov)	Leaf y Composite sustituibles	Decorator sustituible por Component	—
I (Interface Segregation)	Interfaz única para todos	Interfaz única	Interfaz simplificada

Principio	Composite	Decorator	Facade
D (Dependency Inversion)	Cliente depende de Component	Cliente depende de Component	Cliente depende de Facade, no de subsistemas

---

BLOQUE 5 — CIERRE Y ACTIVIDADES

5.1. Preguntas para la reflexión

1. ¿Por qué en Decorator el orden de aplicación puede importar? (Ejemplo: MarcaAgua antes vs. después de Encabezado)
2. ¿Qué diferencia hay entre Facade y un "God Object"? (Facade coordina, no implementa lógica)
3. ¿Puede un Facade también ser Singleton? (Sí, es común: un punto de acceso único)
4. ¿Cómo se relaciona Decorator con el principio Open/Closed? (Nuevos decoradores sin modificar código existente)
5. ¿Qué patrón usarías para modelar la estructura de un plan de estudios? (Composite: Facultad → Programa → Área → Asignatura)

5.2. Próxima sesión

Patrones estructurales restantes:

- **Flyweight:** Optimización de memoria para objetos compartidos
- **Proxy:** Control de acceso a objetos
- **Comparativa final de los siete patrones estructurales**

---

6. REFERENCIAS BIBLIOGRÁFICAS

Bibliografía básica:

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Decorator: pp. 175–184; Facade: pp. 185–193)

- Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2nd ed.). O'Reilly. (Cap. 3: Decorator; Cap. 9: Composite)
- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall. (Cap. 8–12: SOLID)

Webgrafía:

- Refactoring.Guru. (2023). *Decorator*. https://refactoring.guru/design-patterns/decorator
- Refactoring.Guru. (2023). *Facade*. https://refactoring.guru/design-patterns/facade
- SourceMaking. (2023). *Decorator Pattern*. https://sourcemaking.com/design_patterns/decorator
- SourceMaking. (2023). *Facade Pattern*. https://sourcemaking.com/design_patterns/facade

Software y herramientas:

- Java 17 + IntelliJ IDEA/Eclipse
- PlantUML para diagramas de clases
- Git/GitHub para control de versiones
- JUnit 5 para pruebas

---

7. MATERIAL DE APOYO PARA EL DOCENTE

7.1. Errores comunes de los estudiantes

Error	Corrección
Usar Decorator para cambiar interfaz	Decorator **no cambia** la interfaz; para eso está Adapter
No delegar en el componente decorado	Cada decorador debe llamar a componente.operacion()

Error	Corrección
Olvidar el orden de decoración	Documentar el orden cuando el resultado depende de él
Crear decoradores que hacen demasiado	Cada decorador debe tener **una** responsabilidad
Confundir Facade con Adapter	Facade **simplifica**; Adapter **traduce**
Hacer que Facade contenga lógica de negocio	Facade solo **coordina**; la lógica vive en los subsistemas

7.2. Plantilla de ADR para los estudiantes

markdown

```
# ADR 0009: Uso de Decorator y Facade en el Sistema de Gestión Académica

## Estado

Aceptado

## Contexto

[Describir el problema: extensión de reportes y acceso simplificado a subsistemas]

## Alternativas consideradas

1. Herencia pura para reportes (problema: 2^n clases)
2. if/else por tipo de reporte (problema: viola Open/Closed)
3. Cliente conoce todos los subsistemas (problema: acoplamiento alto)
4. Decorator + Facade (elegida)
```

Decisión

[Describir: Decorator para reportes, Facade para el portal del estudiante]

Consecuencias

- Positivas: extensibilidad, desacoplamiento, simplificación

- Negativas: mayor número de clases, orden de decoración importa, riesgo de God Object

Referencias

- Gamma et al. (1994), pp. 175–193.

---

8. RESUMEN DE LA SEMANA 9

Concepto	**Clave**
Decorator (profundización)	Aplicado a reportes académicos (encabezado, firma, marca de agua, notas al margen)
Facade	PortalEstudianteFacade simplifica el acceso a 5 subsistemas
Relación Decorator-Facade	Decorator extiende; Facade simplifica. No compiten, se complementan
Principios SOLID	Decorator y Facade favorecen Open/Closed y Dependency Inversion
Proyecto integrador	Reportes decorados + portal unificado para estudiantes